به‌عنوان یک قانون کلی، اگر کلیت سیستم مشغول باشد، CFS دامنه‌ی محلی هر هسته را فراتر از تعادل بارگذاری نخواهد کرد تا از جریمه‌های دیرکرد حافظه‌ی سیستم‌های NUMA اجتناب شود.

۵-۷-۲ مثال: زمانبندی ویندوز

ویندوز، نخ‌ها را با استفاده از یک الگوریتم زمانبندی الویتی پس‌گرفتنی زمانبندی می‌نماید. زمانبند ویندوز تضمین می‌کند که همیشه نخ با بالاترین الویت اجرا خواهد شد. بخشی از هسته‌ی ویندوز که کار زمانبندی را انجام می‌دهد **اعزام‌کننده** نامیده می‌شود. نخی که توسط اعزام‌کننده برای اجرا انتخاب می‌شود اجرا خواهد داشت تا این که، توسط نخی با الویت بالاتر پس‌گرفته شود، خاتمه بپذیرد، کمیت زمانیش تمام شود و یا با یک فراخوانی سیستمی، مثلاً برای I/O، مسدود شود. در صورتی‌که نخ بلادرنگی با الویت بالاتر برای اجرا آماده شود، در حالی‌که یک نخ با الویت پایین‌تر اجرا دارد، نخ با الویت پایین‌تر پس‌گرفته خواهد شد. با این پس‌گیری، یک نخ بلادرنگ، هر زمان که به دسترسی پردازنده نیاز داشته باشد در دستیابی برتری خواهد داشت.

اعزام‌کننده برای تعیین ترتیب اجرای نخ از یک طرح الویت ۳۲ سطحی استفاده می‌کند. الویت‌ها به دو رده تقسیم می‌شوند. **رده‌ی متغیر** که شامل نخ‌های با الویت ۱ تا ۱۵ و **رده‌ی بلادرنگ** که شامل نخ‌های با الویت ۱۶ تا ۳۱ می‌باشد (یک نخ اجرایی با الویت ۰ هم وجود دارد که برای مدیریت حافظه استفاده می‌شود). اعزام‌کننده برای هر یک از الویت‌های زمانبندی از یک صف استفاده می‌کند و مجموعه‌ی صف‌ها را از بالاترین الویت به پایین‌ترین الویت می‌پیماید تا این که نخ آماده‌ای را برای اجرا بیابد. اگر اعزام‌کننده نخ آماده‌ای نیابد، نخ خاصی به نام **نخ بیکار** را اجرا خواهد کرد.

میان الویت‌های عددی هسته‌ی ویندوز و API ویندوز یک رابطه وجود دارد. API ویندوز، شش رده الویت زیر را که یک پردازه می‌تواند به آنها تعلق داشته باشد، مشخص می‌کند:

- IDLE_PRIORITY_CLASS
- BELOW_NORMAL_PRIORITY_CLASS
- NORMAL_ PRIORITY_CLASS
- ABOVE_NORMAL_PRIORITY_CLASS
- HIGH_PRIORITY_CLASS
- REALTIME_PRIORITY_CLASS

پردازه‌ها نوعاً عضو NORMAL_PRIORITY_CLASS هستند. یک پردازه به این رده تعلق دارد، مگر این که والد پردازه عضوی از IDLE_PRIORITY_CLASS بوده یا رده‌ی دیگری در زمان ایجاد پردازه مشخص شود. به علاوه، رده‌ی الویت یک پردازه می‌تواند با تابع SetPriorityClass() در API ویندوز تغییر داده شود. الویت تمامی رده‌ها به استثنای REALTIME_PRIORITY_CLASS متغیر است، به این معنی که الویت هر یک از نخ‌های متعلق به هر یک از این رده‌ها قابل تغییر است.

هر نخ موجود در یک رده‌ی الویت مشخص، یک الویت نسبی هم دارد. مقادیر الویت‌های نسبی عبارتند از:

- IDLE
- LOWEST
- BELOW_NORMAL
- NORMAL
- ABOVE_NORMAL
- HIGHEST
- TIME_CRITICAL

الویت هر نخ مطابق رده‌ی الویت تعلق آن و الویت نسبی نخ در آن رده تعیین می‌شود. این رابطه در شکل ۵–۲۸ نشان داده می‌شود. مقادیر رده‌های الویت در سطر بالاتر مشخص شده‌اند. ستون سمت چپ شامل مقادیر الویت‌های نسبی است. برای مثال، اگر الویت نسبی یک نخ در رده‌ی ABOVE_NORMAL_PRIORITY_CLASS، NORMAL باشد، الویت عددی آن نخ برابر ۱۰ خواهد بود.

به علاوه، هر نخ یک الویت پایه دارد که مقداری از بازه‌ی الویت رده‌ی تعلق نخ می‌باشد. به‌طور پیش‌فرض، الویت پایه، مقدار الویت نسبی NORMAL برای آن رده است. الویت‌های پایه‌ی هر رده الویت عبارتند از:

- REALTIME_PRIORITY_CLASS—24
- HIGH_PRIORITY_CLASS—13
- ABOVE_NORMAL_PRIORITY_CLASS—10
- NORMAL_PRIORITY_CLASS—8
- BELOW_NORMAL_PRIORITY_CLASS—6
- IDLE_PRIORITY_CLASS—4

الویت اولیه‌ی یک نخ نوعاً الویت پایه‌ی پردازه‌ای است که نخ به آن تعلق دارد، اگرچه تابع SetThreadPriority() در API ویندوز هم می‌تواند برای اصلاح الویت پایه‌ی یک نخ استفاده شود.

	real-time	high	above normal	normal	below normal	idle priority
time-critical	31	15	15	15	15	15
highest	26	15	12	10	8	6
above normal	25	14	11	9	7	5
normal	24	13	10	8	6	4
below normal	23	12	9	7	5	3
lowest	22	11	8	6	4	2
idle	16	1	1	1	1	1

شکل۵–۲۸ الویت نخ‌های ویندوز.

زمانی‌که کمیت زمانی یک نخ به اتمام می‌رسد، نخ متوقف می‌گردد. اگر نخ در رده‌ی الویت متغیر باشد، الویت آن پایین‌تر می‌آید. با این وجود، الویت آن هرگز به پایین‌تر از الویت پایه نمی‌رسد. پایین‌آوردن الویت نخ با این هدف انجام می‌شود که مصرف پردازنده توسط نخ‌های پاپیچ محاسباتی محدود شود. زمانی‌که یک نخ با الویت متغیر از وضعیت انتظار رها می‌شود، اعزام‌کننده الویت آن را افزایش می‌دهد. میزان افزایش بستگی به چیزی دارد که نخ منتظر آن بوده است. برای مثال، نخی که به انتظار صفحه‌کلید بوده است با افزایش زیادی مواجه می‌شود، در

حالی که نخی که منتظر عملیات دیسک بوده، افزایش متوسطی خواهد داشت. این راهبرد می‌خواهد که نخ‌های محاوره‌ای که از ماوس و پنجره‌ها استفاده می‌کنند، زمان‌های پاسخ خوبی ببینند. به این ترتیب، نخ‌های پاپیچ I/O دستگاه‌های I/O را مشغول نگه خواهند داشت، در حالی‌که نخ‌های پاپیچ محاسباتی می‌توانند در پس‌زمینه از دورهای مانده از پردازنده استفاده کنند. این راهبرد توسط چندین سیستم اشتراک‌زمانی، از جمله یونیکس استفاده می‌شود. به علاوه، پنجره‌ای که در حال حاضر با کاربر محاوره دارد، برای بهبود زمان پاسخ خود یک افزایش الویت نیز به‌دست می‌آورد.

زمانی که کاربری در حال اجرای یک برنامه‌ی محاوره‌ای است، سیستم لازم می‌بیند که کارایی خوبی مخصوصاً برای آن پردازه فراهم کند. به همین دلیل، ویندوز برای پردازه‌هایی که در رده‌ی NORMAL_PRIORITY_CLASS قرار دارند قاعده‌ی زمانبندی خاصی دارد. ویندوز، **پردازه‌ی پیش‌زمینه** را که فعلاً بر روی صفحه‌نمایش انتخاب شده از **پردازه‌های پس‌زمینه** که فعلاً انتخاب نشده‌اند، متمایز می‌سازد. زمانی‌که پردازه‌ای به پیش‌زمینه می‌آید، ویندوز کمیت زمانبندی آن را با ضریبی—نوعاً، ۳—افزایش می‌دهد. با این افزایش، پیش از آنکه یک پس‌گیری اشتراک‌زمانی اتفاق بیفتد، طول اجرای پردازه‌ی پیش‌زمینه تا سه برابر افزایش می‌یابد.

ویندوز ۷، **زمانبندی مد کاربر** (**UMS**) را معرفی کرد که به برنامه‌ها اجازه می‌دهد تا نخ‌ها را مستقل از هسته ایجاد و مدیریت کنند. بنابراین، یک برنامه می‌تواند نخ‌های متعددی را بدون این که پای زمانبند هسته‌ی ویندوز در میان بیاید، ایجاد و زمانبندی کند. برای برنامه‌هایی که تعداد بسیار زیادی نخ ایجاد می‌کنند، زمانبندی نخ‌ها در مد کاربر بسیار کارآمدتر از زمانبندی نخ مد هسته است، چرا که هیچ مداخله‌ای از جانب هسته لازم نیست.

نسخه‌های قدیمی‌تر ویندوز، امکان مشابهی تحت عنوان **فیبر** فراهم می‌کردند که به چندین نخ مد کاربر (فیبرها) اجازه می‌داد تا به یک نخ منفرد هسته نگاشته شوند. با این وجود، فیبرها کاربرد عملی محدودی داشتند. فیبر نمی‌توانست فراخوانی‌های API ویندوز را انجام بدهد، چون تمامی فیبرها به ناچار، بلاک محیط نخ (TEB) نخی را که روی آن اجرا داشتند به اشتراک می‌گرفتند. این موضوع در صورتی که یک تابع API ویندوز، اطلاعات وضعیتی یک فیبر را در TEB می‌گذاشت یک مسأله به وجود می‌آورد، چون اطلاعات می‌توانست توسط فیبر دیگری رونویسی شود. UMS با تدارک یک زمینه‌ی خاص برای هر نخ مد کاربر بر این مسأله فایق شد.

به علاوه، برعکس فیبرها، هدف UMS این نبود که به صورت مستقیم توسط برنامه‌نویس استفاده شود. جزئیات نوشتن زمانبندهای مد کاربر می‌تواند با چالش فراوانی مواجه شود و UMS چنین زمانبندی را نمی‌گذارد. در عوض، زمانبندهایی از کتابخانه‌های زبان برنامه‌نویسی سر برآورده‌اند که بالای UMS ساخته شده‌اند. برای مثال، **زمان اجرای همروندی**[1] (ConcRT)، یک چارچوب برنامه‌نویسی همروند برای C++ است که برای موازی‌سازی (بخش ۴-۲) بر روی پردازنده‌های چندهسته‌ای طراحی شده است. ConcRT یک زمانبند مد کاربر به همراه امکاناتی برای تجزیه‌ی برنامه‌ها به وظایف فراهم می‌کند که می‌توانند بر روی هسته‌های پردازشی موجود زمانبندی شوند.

1. Concurrency Runtime

آن‌گونه که در بخش ۵-۵ بیان شد ویندوز از زمان‌بندی بر روی سیستم‌های چندپردازنده ضمن تلاش برای زمان‌بندی یک نخ بر روی بهترین هسته‌ی پردازشی برای آن نخ که مشمول حفظ پردازنده‌ی ترجیحی یک نخ در کنار اخیرترین پردازنده‌ی آن پشتیبانی می‌کند. یک تکنیک که توسط ویندوز استفاده می‌شود ایجاد مجموعه‌هایی از پردازنده‌های منطقی است (معروف به **مجموعه‌های SMT**). در یک سیستم SMT اَبَرنخی، نخ‌های سخت‌افزاری متعلق به یک هسته‌ی CPU یکسان به مجموعه‌ی SMT یکسان نیز تعلق دارند. پردازنده‌های منطقی با شروع از ۰ شماره‌گذاری شده‌اند. به‌عنوان یک مثال، یک سیستم دونخی/چهارهسته‌ای شامل هشت پردازنده‌ی منطقی، حاوی چهار مجموعه‌ی SMT خواهد بود: {0, 1}، {2, 3}، {4, 5}، و {6, 7}. برای اجتناب از جریمه‌های دسترسی حافظه که در بخش ۵-۵-۴ تأکید شد، زمان‌بند تلاش می‌کند تا یک نخ در حال اجرا بر روی پردازنده‌های منطقی در مجموعه‌ی SMT یکسان را حفظ کند.

برای توزیع بارها در میان پردازنده‌های منطقی متفاوت، هر نخ به یک **پردازنده‌ی ایده‌آل** گمارده می‌شود که یک شماره، معرف پردازنده‌ی ترجیحی نخ می‌باشد. هر پردازه یک دانه مقدار[1] اولیه دارد که پردازنده‌ی ایده‌آل یک نخ متعلق به آن پردازه را مشخص می‌کند. این دانه به‌ازای هر نخ جدیدی که توسط پردازه ایجاد می‌شود یک‌واحد افزایش می‌یابد که با این کار، بار در میان پردازنده‌های منطقی متفاوتی توزیع می‌شود. در سیستم‌های SMT، واحد افزایشی برای پردازنده‌ی ایده‌آل بعدی در مجموعه‌ی SMT بعدی است. برای مثال، روی یک سیستم دونخی/چهارهسته‌ای، پردازنده‌ی ایده‌آل نخ‌ها در یک پردازه‌ی به‌خصوص مقادیر ۰، ۲، ۴، ۶، ۰، ۲، ... را خواهند داشت. برای اجتناب از شرایطی که به نخ اول هر پردازه پردازنده‌ی ۰ گمارده نشود، به پردازه‌ها دانه مقادیر متفاوتی نسبت داده می‌شود که با این کار، بار نخ‌ها در میان تمامی هسته‌های پردازشی فیزیکی در سیستم توزیع می‌شود. در تداوم مثال بالایی ما، اگر دانه برای پردازه‌ی دومی ۱ باشد، پردازنده‌های ایده‌آل به‌ترتیب به مقادیر ۱، ۳، ۵، ۷، ۱، ۳، و الی‌آخر گمارده خواهند شد.

۵-۷-۳ مثال: زمانبندی سولاریس

سولاریس از زمانبندی نخ مبتنی بر الویت استفاده می‌کند. هر یک از نخ‌ها به یکی از شش رده‌ی زیر تعلق دارند:

۱- اشتراک‌زمانی (TS)
۲- محاوره‌ای (IA)
۳- بلادرنگ (RT)
۴- سیستمی (SYS)
۵- سهم منصفانه (FSS)
۶- الویت ثابت (FP)

در میان هر یک از این رده‌ها، الویت‌ها و الگوریتم‌های زمانبندی متفاوتی وجود دارند.

1. Seed value

رده‌ی پیش‌فرض زمانبندی یک پردازه، اشتراک‌زمانی است. سیاست زمانبندی رده‌ی اشتراک‌زمانی، الویت‌ها را به صورت پویا تغییر داده و با استفاده از یک صف با بازخورد چندسطحی، برش‌های زمانی با طول‌های متفاوتی را نسبت می‌دهد. به صورت پیش‌فرض، بین الویت‌ها و برش‌های زمانی یک رابطه‌ی معکوس وجود دارد. الویت بالاتر، برش زمانی کوچکتر؛ و الویت پایین‌تر، برش زمانی بزرگ‌تری دارد. پردازه‌های محاوره‌ای نوعاً از الویت بالاتر و پردازه‌های پاپیچ پردازنده از الویت پایین‌تری برخوردارند. این سیاست زمانبندی، زمان پاسخ بهتری به پردازه‌های محاوره‌ای و بازده بهتری برای پردازه‌های پاپیچ پردازنده به بار می‌آورد. سیاست زمانبندی رده‌ی محاوره‌ای با رده‌ی اشتراک‌زمانی یکسان است، اما در آن برای کارایی بهتر به برنامه‌های کاربردی پنجره‌ای— نظیر آنهایی که توسط مدیر پنجره‌های KDE یا GNOME ایجاد می‌شوند—الویت بالاتری داده می‌شود.

شکل ۵-۲۹، جدول اعزام ساده شده‌ی زمانبندی نخ‌های اشتراک‌زمانی و محاوره‌ای را نشان می‌دهد. این دو رده‌ی زمانبندی، دارای ۶۰ سطح الویت هستند که برای اختصار فقط چند مورد از آنها را نمایش می‌دهیم. (برای مشاهده‌ی جدول اعزام کامل یک سیستم سولاریس یا VM، فرمان dispadmin –c TS –g را اجرا کنید.) جدول اعزام نشان داده شده در شکل ۵-۲۹، شامل فیلدهای زیر است:

- **الویت**: الویت وابسته به رده برای رده‌های اشتراک‌زمانی و محاوره‌ای. عدد بالاتر نشانگر الویت بالاتر است.
- **کمیت زمانی**: کمیت زمانی مربوط به الویت. یک رابطه‌ی معکوس میان الویت‌ها و کمیت‌های زمانی وجود دارد: پایین‌ترین الویت (۰)، بالاترین کمیت زمانی (۲۰۰ میلی‌ثانیه) و بالاترین الویت (۵۹)، پایین‌ترین کمیت زمانی (۲۰ میلی‌ثانیه) را دارد.

priority	time quantum	time quantum expired	return from sleep
0	200	0	50
5	200	0	50
10	160	0	51
15	160	5	51
20	120	10	52
25	120	15	52
30	80	20	53
35	80	25	54
40	40	30	55
45	40	35	56
50	40	40	58
55	40	45	58
59	20	49	59

شکل ۵-۲۹ جدول اعزام نخ‌های اشتراک‌زمانی و محاوره‌ای سولاریس.

- **انقضای کمیت زمانی**: الویت جدید نخی که کل کمیت زمانی خود را بدون انسداد

استفاده کرده است. این قبیل نخ‌ها پاپیچ پردازنده محسوب می‌شوند. همان‌گونه که در جدول نشان داده می‌شود، این نخ‌ها الویت‌های پایین‌تری برای خود می‌بینند.

- **برگشت از خواب**: الویت نخی که از خواب (نظیر انتظار I/O) بیدار می‌شود. همان‌گونه که جدول نشان می‌دهد، زمانی‌که برای نخ منتظری I/O فراهم است، الویت آن بین ۵۰ تا ۵۹ افزایش می‌یابد، با این پشتیبانی سیاست زمانبندی، زمان پاسخ مناسبی برای پردازه‌های محاوره‌ای فراهم می‌شود.

بالاترین الویت به نخ‌های رده‌ی بلادرنگ داده می‌شود. پردازه‌ی بلادرنگ قبل از هر پردازه‌ی دیگری از سایر رده‌ها اجرا خواهد داشت. این انتساب به یک پردازه‌ی بلادرنگ امکان می‌دهد تا در یک دوره‌ی زمانی مشخص، پاسخ تضمین‌شده‌ای از سیستم دریافت کند. با این وجود، کلاً پردازه‌های کمی به رده‌ی بلادرنگ تعلق دارند.

سولاریس برای اجرای نخ‌های هسته، نظیر زمانبند و گماشته‌ی صفحه‌بندی از رده‌ی سیستمی استفاده می‌کند. همین که الویت یک نخ سیستمی مشخص شد، تغییر نمی‌کند. رده‌ی سیستمی برای استفاده‌ی هسته محفوظ است (آن دسته از پردازه‌های کاربر که در مد هسته اجرا دارند جزء رده‌ی سیستمی محسوب نمی‌شوند).

رده‌های الویت ثابت و سهم منصفانه با سولاریس ۹ معرفی شدند. بازه‌ی الویت نخ‌های رده‌ی الویت ثابت با رده‌ی اشتراک‌زمانی یکی است؛ با این وجود، الویت‌های آنها به صورت پویا تنظیم نمی‌شود. رده‌ی سهم منصفانه در تصمیم‌گیری زمانبندی به جای الویت‌ها از سهام پردازنده استفاده می‌کند. سهام پردازنده نمود منابع پردازنده‌ی موجود بوده و در اختیار مجموعه‌ای از پردازه‌ها (تحت عنوان **پروژه**) قرار می‌گیرد.

هر رده‌ی زمانبندی شامل مجموعه‌ای از الویت‌هاست. با این وجود، زمانبند، الویت‌های خاص رده را به الویت‌های سراسری تبدیل کرده و نخی را برای اجرا انتخاب می‌کند که بالاترین الویت سراسری را داشته باشد. نخ انتخابی در پردازنده اجرا می‌شود تا این که (۱) مسدود شود، (۲) برش زمانی خود را استفاده کند، یا (۳) توسط یک نخ با الویت بالاتر پس‌گرفته شود. اگر چندین نخ با الویت یکسان موجود باشند، زمانبند از صف نوبت-چرخشی استفاده می‌کند. شکل ۵-۳۰، چگونگی ارتباط شش رده‌ی زمانبندی را با یکدیگر و نگاشت آنها به الویت‌های سراسری را نشان می‌دهد. توجه داشته باشید که هسته، ۱۰ نخ را برای سرویس‌دهی وقفه‌ها نگه می‌دارد. این نخ‌ها به هیچ یک از رده‌های زمانبندی تعلق نداشته و با بالاترین الویت (۱۶۰-۱۶۹) اجرا می‌شوند. همان‌گونه که عنوان شد، سولاریس در سابق از مدل چند-به-چند (بخش ۴-۳-۳) استفاده می‌کرد، اما با آمدن سولاریس ۹، مدل یک-به-یک (بخش ۴-۳-۲) را انتخاب کرد.

۵-۸ ارزیابی الگوریتم

چگونه الگوریتم زمانبندی پردازنده را برای یک سیستم خاص انتخاب کنیم؟ همان‌گونه که در بخش ۵-۳ دیدیم، الگوریتم‌های زمانبندی زیادی با پارامترهای خاص خود وجود دارند. در نتیجه، انتخاب یک الگوریتم می‌تواند مشکل باشد.